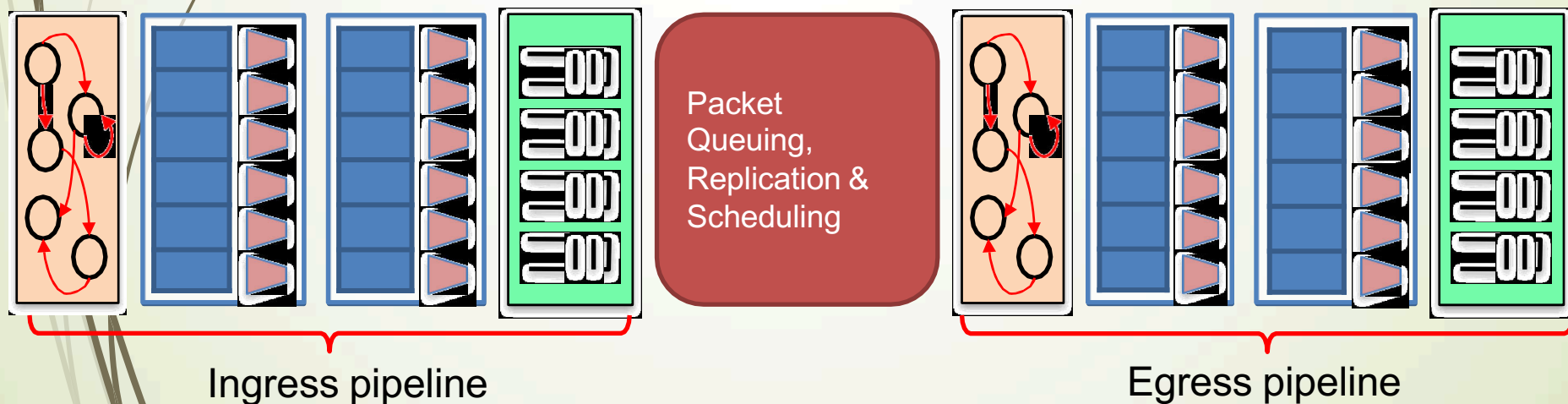


Alternative Switch Model

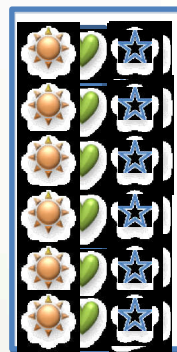
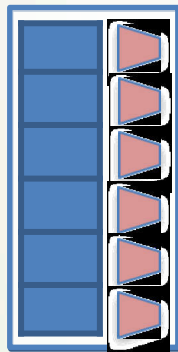
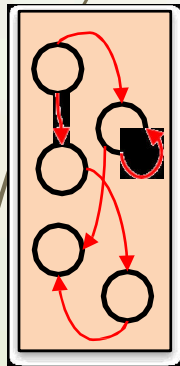
- **Symmetric Ingress and Egress Pipelines**
- **Fully and independently programmable parsers and deparsers**



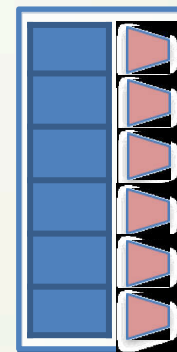
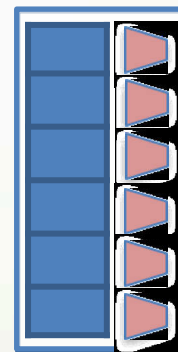
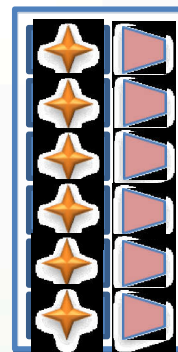
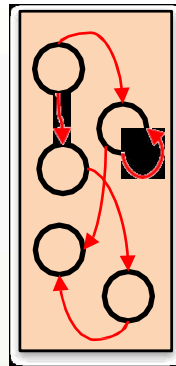
Extended Pipeline Model

- **“Non-standard” combination of programmable components**
 - Two parsers can be very useful for tunnel processing
 - Many other combinations are also possible
- **Specialized components available for advanced functionality**

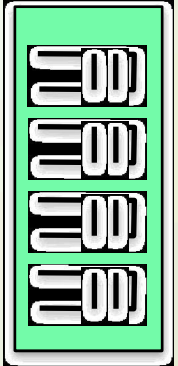
Programmable
Parser 1



Programmable
Parser 2

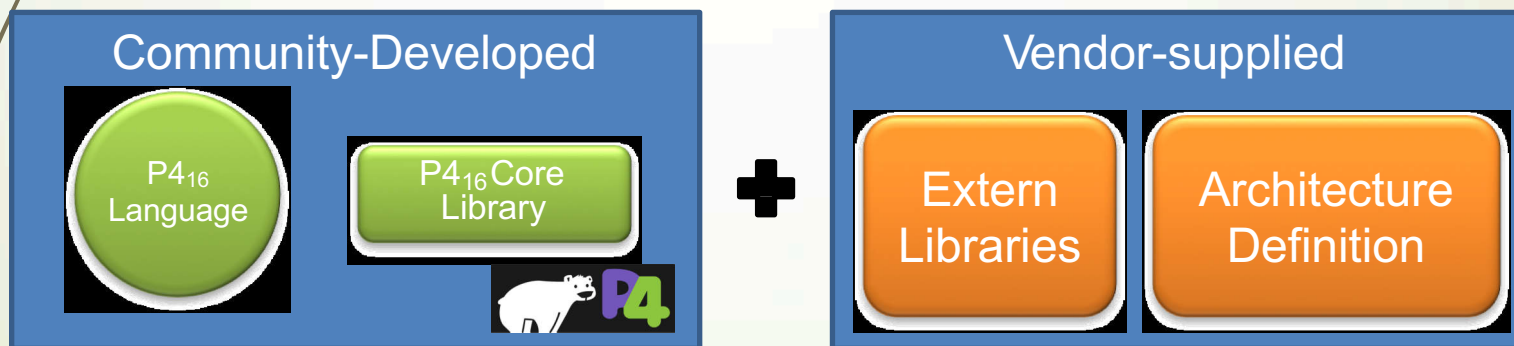


Programmable
Deparser



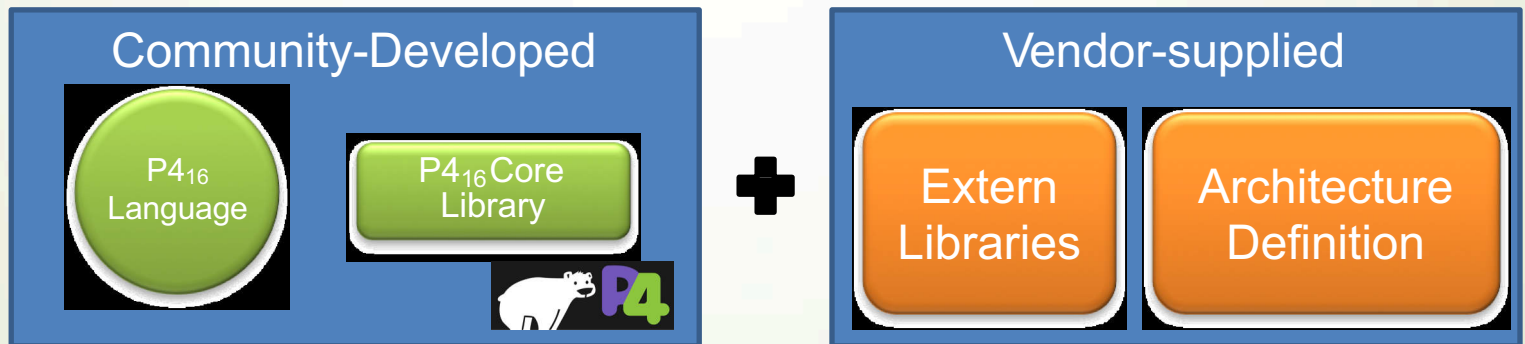
The New P4₁₆ Approach

Term	Explanation
P4 Target	An embodiment of a specific hardware implementation
P4 Architecture	A specific set of P4-programmable components , externs, fixed components and their interfaces available to the P4 programmer
P4 Platform	P4 Architecture implemented on a given P4 Target



P4_16 Program Portability

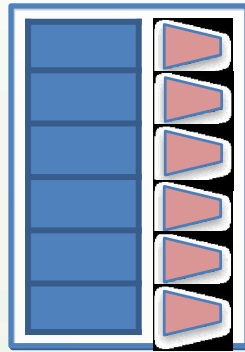
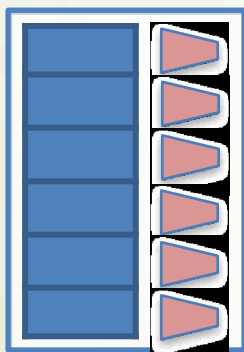
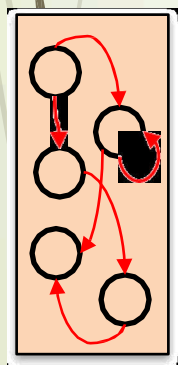
P4 programs	C/C++ programs
Target-Independent	Generally portable across a wide variety of CPUs
Architecture-Dependent	Generally not portable when non-standard libraries or OS-specific features are used
Targets can implement multiple architectures	Multiple Operating Systems can be implemented for a given CPU
Multiple targets can support the same architecture	A portable Operating System or library can run on different CPUs



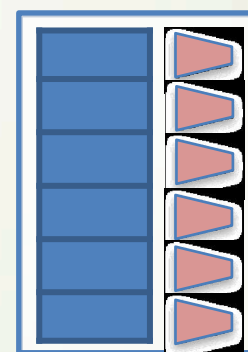
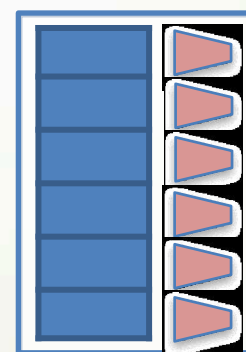
V1 Architecture

- **Compatible with P4_14 architecture**
- **Is implemented on top of `bmv2-simple_switch` target**
- **Will be gradually introduced in the due course**

Programmable
Parser



Packet
Queuing,
Replication &
Scheduling

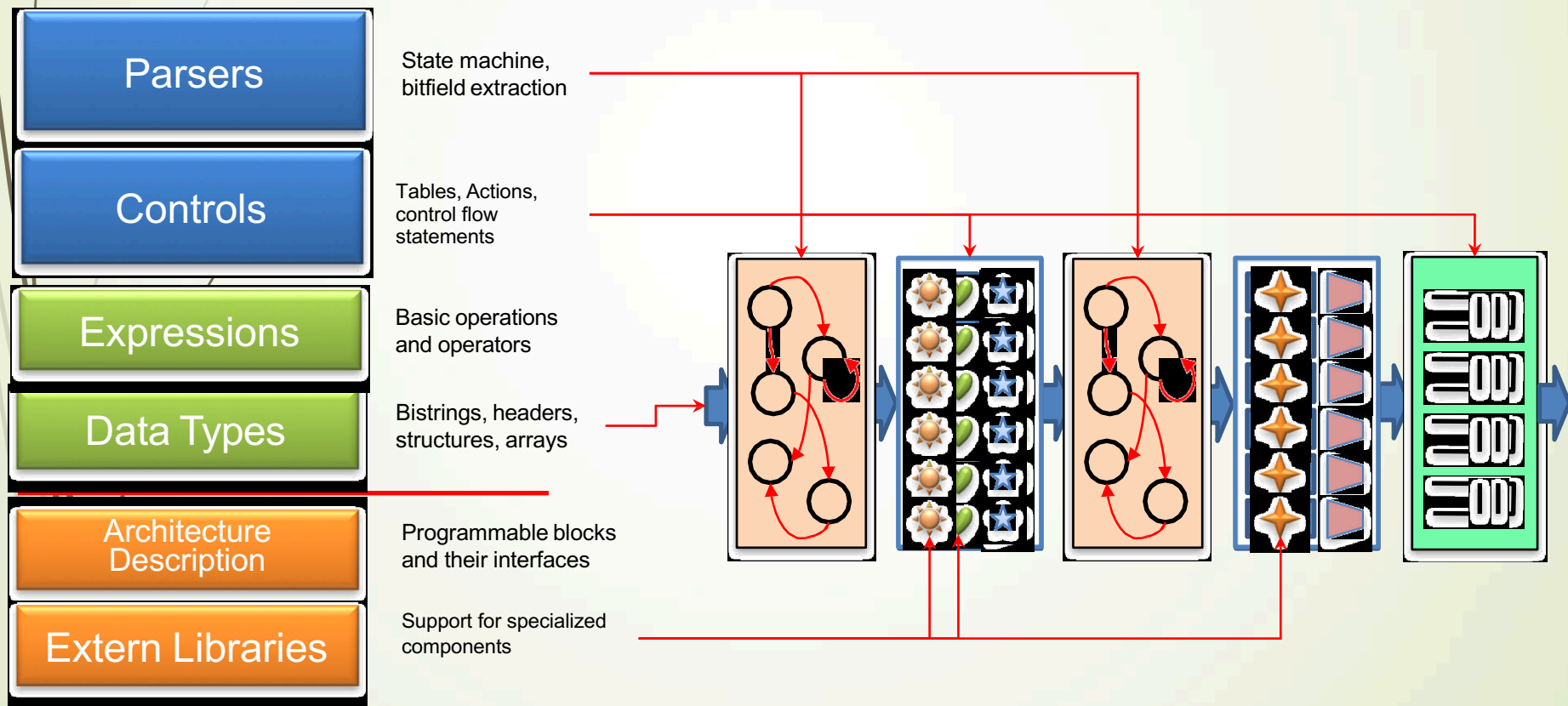


Programmable
Deparser



P4₁₆ Language Elements

We will quickly skim through them



Agenda

- Why Dataplan Programming
- P4 Overview
- **Main Data Types**
- Packet Parsing
- Packet Processing
- Packet Deparsing
- Program Structure
- P4 Runtime
- Summary

Sample Header Definitions

Example: Declaring L2 headers

```
header ethernet_t {
    bit<48>    dstAddr;
    bit<48>    srcAddr;
    bit<16>    etherType;
}

header vlan_tag_t {
    bit<3>     pri;
    bit<1>     cfi;
    bit<12>    vid;
    bit<16>    etherType;
}

struct my_headers_t {
    ethernet_t  ethernet;
    vlan_tag_t[2] vlan_tag;
}
```

- **Basic Types**

- **bit<n>** – Unsigned integer (bitstring) of length n
 - **bit** is the same as **bit<1>**
- **int<n>** – Signed integer of length n (>=2)
- **varbit<n>** – Variable-length bitstring

- **Derived Types**

- **header** – Ordered collection of members
 - Byte-aligned
 - Can be valid or invalid
 - Can contain bit<n>, int<n> and varbit<n>
- **struct** – Unordered collection of members
 - No alignment restrictions
 - Can contain any basic or derived types
- Header Stacks -- arrays of headers

Using structs for Intrinsic Metadata

```
typedef bit<9> port_id_t; /* Switch port */
typedef bit<16> mgid_t; /* Multicast Group */
typedef bit<5> id_t; /* Queue ID */

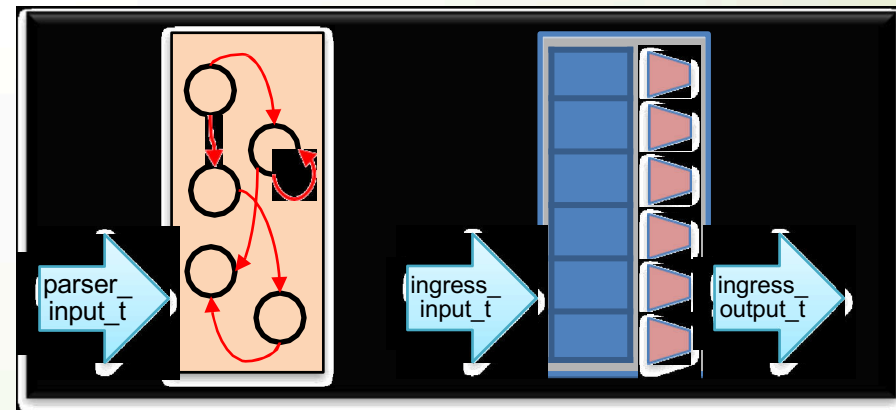
struct parser_input_t {
    port_id_t ingress_port;
    bit<1> resubmit_flag;
}

struct ingress_input_t {
    portid_t ingress_port;
    timestamp_t ingress_timestamp;
}

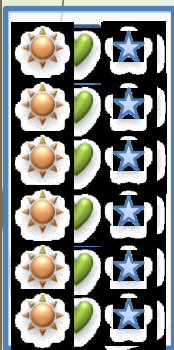
struct ingress_output_t {
    portid_t egress_port;
    mgid_t mcast_group;
    bit<1> drop_flag;
    qid_t egress_queue;
}
```

- Intrinsic Metadata is the data that a P4-programmable components can use to interface with the **rest of the system**
- These definitions come from the files, **supplied by the vendor**

P4 Platform



The Need for Externs



- **Most platforms contain specialized facilities**
 - They differ from vendor to vendor
 - They can't be expressed in the core language
 - Specialized computations
 - They might have control-plane accessible state or configuration
- **The language should stay the same**
 - In P4₁₄ almost 1/3 of all the constructs were dedicated to specialized processing
 - In P4₁₆ all specialized objects use the same interface
- **Objects can be used even if their implementation is hidden**
 - Through instantiation and method calling

Stateless and Stateful Objects

- **Stateless Objects: Reinitialized for each packet**
 - Variables (metadata), packet headers, packet_in, packet_out
- **Stateful Objects: Keep their state between packets**
 - Tables
 - Externs
 - P4₁₄: Counters, Meters, Registers, Parser Value Sets, Selectors, etc.

Object	Data Plane Interface		Control Plane Can	
	Read State	Modify/Write State	Read	Modify/Write
Table	apply()	---	Yes	Yes
Parser Value Set	get()	---	Yes	Yes
Counter	---	count()	Yes	Yes*
Meter	execute ()		Configuration Only	Configuration Only
Register	read()	write()	Yes	Yes

Meters in V1 Architecture

Definition

/* Definition in vlmodel.p4 */

```
enum MeterType {
    packets,
    bytes
}
```

```
extern meter {
    meter(bit<32> instance_count, MeterType type);
    void execute_meter<T>(in bit<32> index, out T result);
}
```

This is a template definition. The method will accept the parameter of any type

Color Coding:

1 – Green
2 – Yellow
3 – Red

Usage

```
typedef bit<2> meter_color_t;

const meter_color_t METER_COLOR_GREEN = 0;
const meter_color_t METER_COLOR_YELLOW = 1;
const meter_color_t METER_COLOR_RED = 2;

meter(1024, MeterType.bytes) acl_meter;

action color_my_packets(bit<10> index) {
    acl_meter.execute_meter((bit<32>)index, meta.color);
}

table acl {
    key = { . . . }
    actions = { color_my_packets; . . . }
}

apply {
    acl.apply();
    if (meta.color == METER_COLOR_RED) {
        mark_to_drop();
    }
}
```

Hashes and Random Numbers – Stateless Externs

IPv4 Checksum Verification

```
control MyVerifyChecksum(in    my_headers_t  hdr,
                        inout my_metadata_t meta)
{
    Checksum16() ipv4_checksum;
    bit<16>      ck;

    apply {
        if (hdr.ipv4.isValid()) {
            ck = ipv4_checksum.get(
                {
                    hdr.ipv4.version,      hdr.ipv4.ihl,
                    hdr.ipv4.diffserv,      hdr.ipv4.totalLen,
                    hdr.ipv4.identification, hdr.ipv4.flags,
                    hdr.ipv4.fragOffset,    hdr.ipv4.ttl,
                    hdr.ipv4.protocol,      hdr.ipv4.srcAddr,
                    hdr.ipv4.dstAddr
                });
            if (hdr.ipv4.hdrChecksum != ck) {
                mark_to_drop();
            }
        }
    }
}
```

IPv4 Checksum Update

```
control MyComputeChecksum(inout my_headers_t  hdr,
                          inout my_metadata_t meta)
{
    Checksum16() ipv4_checksum;

    apply {
        if (hdr.ipv4.isValid()) {
            hdr.ipv4.hdrChecksum = ipv4_checksum.get(
                {
                    hdr.ipv4.version,      hdr.ipv4.ihl,
                    hdr.ipv4.diffserv,      hdr.ipv4.totalLen,
                    hdr.ipv4.identification, hdr.ipv4.flag,
                    hdr.ipv4.fragOffset,    hdr.ipv4.ttl,
                    hdr.ipv4.protocol,      hdr.ipv4.srcAddr,
                    hdr.ipv4.dstAddr
                });
        }
    }
}
```

Agenda

- Why Dataplan Programming
- P4 Overview
- Main Data Types
- Packet Parsing
- Packet Processing
- Packet Deparsing
- Program Structure
- P4 Runtime
- Summary